

Aufbau des Source Trees

Subsysteme, Schnittstellen und Build-Strukturen im Linux-Kernel verstehen

1.08

Die Root-Dateien geben dem Source Tree Struktur

README

Einstieg, Projektüberblick und Dokumentationspfade

Makefile

Globaler Build-Einstieg und zentrale Build-Ziele

Kconfig

Startpunkt der konfigurierbaren Kernel-Optionen

Kbuild

Ergänzende Regeln des Kernel-Build-Systems

MAINTAINERS

Zuordnung von Subsystemen, Dateien und Zuständigkeiten

COPYING

Gesamtlizenz und Hinweise zur Lizenzierung

Top-Level-Verzeichnisse spiegeln Verantwortungen

VEREINFACHTE AUSWAHL

— arch/	Architekturabhängiger Code
— block/	Block-I/O-Subsystem
— certs/	Zertifikate und Signaturprüfung
— crypto/	Krypto Algorithmen und APIs
— Documentation/	Kernel-Dokumentation
— drivers/	Gerätetreiber
— fs/	Dateisysteme und VFS
— include/	Headerdateien und Kernel-APIs
— init/	Kernel-Initialisierung und Boot
— io_uring/	Asynch I/O-Schnittstelle io_uring
— ipc/	Inter Process Communication
— kernel/	Zentrale Kernelmechanismen
— lib/	Allgemeine Hilfsbibliotheken
— mm/	Speicherverwaltung
— net/	Netzwerk-Stack
— rust/	Rust-Unterstützung im Kernel
— samples/	Beispielcode

01 Funktion vor Produkt

Subsysteme werden nach Aufgaben organisiert, nicht nach einzelnen Geräten.

02 Auswahl statt Vollständigkeit

Weitere Root-Bereiche sind unter anderem block, crypto, security, rust und io_uring.

Top-Level-Verzeichnisse spiegeln Verantwortungen

VEREINFACHTE AUSWAHL

— scripts/	Build- und Hilfsskripte
— security/	Sicherheitsframeworks und LSM
— sound/	Audio-Subsystem
— tools/	Userspace-Werkzeuge
— usr/	Initiales Userspace-/initramfs
— virt/	Virtualisierung
— Kbuild	Zentrale Build-Regeln
— Kconfig	Zentrale Kernel-Konfiguration
— Makefile	Haupt-Makefile des Kernels
— README	Infos zum Kernel-Sourcetree

01 Funktion vor Produkt

Subsysteme werden nach Aufgaben organisiert, nicht nach einzelnen Geräten.

02 Auswahl statt Vollständigkeit

Weitere Root-Bereiche sind unter anderem block, crypto, security, rust und io_uring.

arch/ bündelt plattformspezifischen Kernelcode

PROZESSORFAMILIE

arch/x86/

Startcode, Exceptions, MMU-Details und x86-spezifische Infrastruktur.

EMBEDDED UND ARM

arch/arm64/

64-Bit-ARM, SoC-Integration und plattformspezifische Device Trees.

WEITERE ARCHITEKTUR

arch/riscv/

RISC-V-spezifische Implementierungen, Header und Bootpfade.

Gemeinsame Subsysteme bleiben außerhalb von arch/ und werden von mehreren Architekturen genutzt.

init/ und kernel/ decken Start und Kernlogik ab

EINSTIEGSPUNKTE

```
init/main.c  
kernel/sched/  
kernel/irq/  
kernel/locking/  
kernel/time/  
kernel/trace/
```

01 init/

Initialisierung, Startablauf und Übergang in den laufenden Kernel.

02 kernel/

Scheduler, Interruptverwaltung, Zeitfunktionen und weitere zentrale Mechanismen.

Die Verzeichniszuordnung zeigt den Einstieg – der tatsächliche Kontrollfluss reicht weiter.

Speicher und I/O verteilen sich auf mehrere Bereiche

mm/

Virtueller Speicher, Page Tables, Reclaim und Speicherallokation

block/

Block-I/O, Request-Verarbeitung und Speichergeräte-Infrastruktur

fs/

VFS, Dateisysteme und Operationen auf Dateien

io_uring/

Asynchrone I/O-Mechanismen und deren Kernelintegration

drivers/ organisiert Hardware nach Geräteklassen

TREIBERBEREICHE

```
drivers/  
├─ gpio/      GPIO-Controller  
├─ i2c/       I2C-Controller  
├─ spi/       SPI-Controller  
├─ net/       Netzwerkadapter  
├─ usb/       USB-Infrastruktur  
├─ input/     Eingabegeräte  
├─ gpu/       Grafik und Display  
└─ hwmon/     Hardware-Monitoring
```

01 Subsystem suchen

Die technische Gerätekategorie ist meist der bessere Startpunkt als der Produktname.

02 Framework mitlesen

Treiber verwenden gemeinsame APIs, Header, Kconfig-Optionen und Build-Regeln.

Weitere Subsysteme besitzen eigene Root-Bereiche

net/

Protokolle, Sockets und Netzwerk-Stack

security/

Sicherheitsmechanismen und LSM-Anbindung

crypto/

Kryptografische APIs und Implementierungen

sound/

Audio-Subsysteme und zugehörige Treiber

virt/

Virtualisierungsbezogene Infrastruktur

rust/

Rust-Infrastruktur und Kernel-Abstraktionen

Header trennen interne APIs von User-Space-UAPI

KERNEL-INTERN

include/linux/

Gemeinsame interne Typen, Kernel-APIs und Schnittstellen zwischen Subsystemen.

USER-SPACE-VERTRAG

include/uapi/

Header für exportierte User-Space-Schnittstellen, etwa `include/uapi/linux/gpio.h`.

ARCHITEKTUR UND BUILD

arch/.../include/

Architekturabhängige Header; generierte Header entstehen zusätzlich im Build-Output.

`make headers_install` exportiert geeignete Kernel-Header für die Nutzung im User Space.

Device Trees verbinden Board, Hardware und Treiber

EMBEDDED-ORIENTIERUNG

```
arch/arm64/boot/dts/  
Documentation/devicetree/bindings/  
drivers/gpio/  
include/linux/gpio/driver.h  
  
$ git grep 'gpio-controller' -- \  
    arch/arm64/boot/dts/
```

01 Board und SoC

DTS/DTSI beschreiben die Hardware; Bindings legen das erwartete Schema fest.

02 Implementierung

Passende Subsystemtreiber und interne Header liegen in eigenen Bereichen.

Hardwarebeschreibung und Treibercode sind verbunden, aber bewusst getrennt abgelegt.

Kconfig definiert Optionen und ihre Abhängigkeiten

VEREINFACHTES BEISPIEL

```
# drivers/gpio/Kconfig
config EXAMPLE_GPIO
    tristate "Example GPIO controller"
    depends on GPIOLIB

# erzeugte .config
CONFIG_EXAMPLE_GPIO=m
```

01 Kconfig beschreibt

Optionen, Typen und Abhängigkeiten definieren zulässige Konfigurationen.

02 .config entscheidet

y baut fest ein, m erstellt ein Modul, n deaktiviert die Option.

Der vollständige Source Tree ist sichtbar – gebaut wird nur die konfigurierte Auswahl.

Kbuild übersetzt Optionen in konkrete Build-Ziele

MAKEFILE / KBUILD

```
# drivers/gpio/Makefile
obj-$(CONFIG_EXAMPLE_GPIO) += example.o

CONFIG_EXAMPLE_GPIO=y   -> eingebaut
CONFIG_EXAMPLE_GPIO=m   -> example.ko
CONFIG_EXAMPLE_GPIO=n   -> kein Build
```

01 obj-y

Objekte für den fest eingebauten Kernel werden gesammelt und mit vmlinux verlinkt.

02 obj-m

Modulobjekte werden als ladbare Kernelmodule gebaut.

Kconfig legt Möglichkeiten fest; Kbuild verknüpft die gewählte Option mit Quelldateien.

Source Tree und Build-Output lassen sich trennen

SEPARATES BUILD-VERZEICHNIS

```
$ make O=/tmp/linux-build defconfig  
$ make O=/tmp/linux-build -j "$(nproc)"
```

```
/tmp/linux-build/.config
```

```
/tmp/linux-build/include/generated/
```

```
/tmp/linux-build/vmlinux
```

```
/tmp/linux-build/Module.symvers
```

01 Saubere Quellen

Generierte Dateien und Build-Produkte liegen im angegebenen Ausgabeordner.

02 O= durchgehend

Dasselbe Output-Verzeichnis muss zu allen zusammengehörenden make-Aufrufen passen.

Welche Artefakte entstehen, hängt zusätzlich von Architektur, Konfiguration und Build-Ziel ab.

Dokumentation, Werkzeuge und Tests gehören dazu

Documentation/

API-Beschreibungen, Subsystem-Dokumentation und Bindings

scripts/

Build-Helfer, Analysewerkzeuge und Kernel-Skripte

tools/

User-Space-Werkzeuge, perf sowie Testinfrastruktur

tools/testing/

KUnit-Werkzeuge und Kselftests in eigenen Unterverzeichnissen

Praxis: Einen GPIO-Pfad durch den Source Tree lesen

01 Hardware und Binding suchen

arch/arm64/boot/dts/ und Documentation/devicetree/bindings/

02 Treiber und interne API finden

drivers/gpio/ sowie include/linux/gpio/driver.h

03 Konfiguration und Build prüfen

drivers/gpio/Kconfig, drivers/gpio/Makefile und die aktive .config

04 User-Space-Schnittstelle zuordnen

include/uapi/linux/gpio.h und /dev/gpiochipX

Analysepfad: Hardwarebeschreibung → Treiber → Build-Auswahl → User-Space-API.